NETWORK VISUALIZER

Project Documentation Package

Corporate software development baseline

Field	Value
Document ID	NV-PKG-001
Product	Network Visualizer
Platform	Windows 10 / 11 (x64)
Stack	Tauri 2 · React · TypeScript · Rust · Npcap
Version	1.0 — Documentation Baseline
Date	2026-08-10
Status	Approved for implementation
Classification	Internal / Open Source candidate

Includes: Project Proposal · Software Requirements Specification · System Architecture & Design · Project Plan & Roadmap · Risk, Security & Privacy

1. Document Control

This package consolidates the enterprise documentation set for Network Visualizer. Markdown sources of truth live under /docs; this file is the stakeholder-facing export.

1.1 Document map

ID	Title	Audience
NV-PROP-001	Project Proposal & Business Case	Sponsors, product, eng
NV-SRS-001	Software Requirements Specification	Product, eng, QA
NV-SAD-001	System Architecture & Design	Engineering, architecture
NV-PLAN-001	Project Plan & Roadmap	PM, engineering
NV-RISK-001	Risk, Security & Privacy	Security, compliance

1.2 Revision history

Ver	Date	Author	Notes
1.0	2026-08-10	Project team	Initial approved baseline

2. Project Proposal & Business Case (NV-PROP-001)

2.1 Executive summary

Network Visualizer is a Windows desktop application that answers: where does my network traffic go, and which application is sending it? The product captures host traffic, attributes flows to processes, geolocates remote endpoints, and renders connections as animated arcs on a 3D interactive globe inside a premium operations-center dashboard.

Recommendation: Proceed to build Phases 0–5 using Tauri 2 + React + TypeScript, Rust/Npcap full traffic capture, MaxMind GeoLite2, and react-globe.gl visualization.

2.2 Problem statement

Pain	Today’s experience
Opaque destinations	Users see Connected but not where on Earth
Tool fragmentation	Task Manager, Wireshark, DevTools each show a slice
Cognitive load	Raw IP tables are accurate but not intuitive
Privacy anxiety	Users want local insight without cloud PCAP upload

2.3 Goals (v1)

1. Visualize active host network flows on a world globe (home → destination).
2. Attribute flows to process name / PID where OS APIs allow.
3. Measure approximate throughput via full traffic capture.
4. Deliver a polished dark ops dashboard UX with smooth animation.
5. Operate local-first with no mandatory cloud telemetry.

2.4 Non-goals (v1)

- Multi-host / fleet monitoring
- Full packet payload inspection or HTTPS MITM
- Official Linux/macOS shipping builds
- Commercial GeoIP accuracy SLAs
- Intrusion detection / automated threat scoring

2.5 Target users

Persona	Need	Success moment
Power user	What is my PC talking to?	Chrome arcs to CDNs appear
Developer	Debug outbound calls	Filter by process, pin a flow
Home lab / small IT	Spatial overview	Top countries strip answers chatty apps
Educator	Teach networking	Globe makes TCP/UDP geo tangible

2.6 Solution overview

Layer	Technology	Role
Desktop shell	Tauri 2	Native window, IPC, packaging
UI	React 18 + TS + Vite	Dashboard and state
Globe	react-globe.gl	3D Earth, arcs, animation
Capture	Npcap + Rust pcap	Full traffic capture
Process map	Windows IP Helper	Socket → PID → name
Geo	GeoLite2-City offline	IP → lat/lon/city/country
Transport	Tauri events 250–500 ms	Delta snapshots to UI

2.7 Success criteria

1. Demo-ready wow within 30 seconds after setup on clean Windows 11.
2. Real browsing produces geo arcs with process labels.
3. Core filters and pause work without deep training.
4. Documented install path; GitHub README sufficient for contributors.
5. No cloud telemetry by default; privacy model documented.

3. Software Requirements Specification (NV-SRS-001)

3.1 Purpose and scope

This SRS defines functional and non-functional requirements for Network Visualizer v1: single-machine Windows client, local capture, process enrichment, offline GeoIP, 3D visualization, dashboard UX, and packaging.

3.2 Capture requirements

ID	Requirement	Pri
CAP-01	Capture IP TCP/UDP via Npcap on selected interface	Must
CAP-02	Parse headers only; no payload storage by default	Must
CAP-03	IPv4 required; IPv6 best-effort	Must
CAP-04	Aggregate into 5-tuple flows with counters	Must
CAP-05	Compute approximate throughput rates	Must
CAP-06	Expire idle flows (default 30–60s)	Must
CAP-07	Detect missing Npcap with remediation UX	Must
CAP-08	Detect lack of elevation with relaunch guidance	Must
CAP-09	List/select capture interfaces in Settings	Should
CAP-10	Degrade to connection-table mode if capture fails	Should

3.3 Process, geo, visualization (summary)

Area	Key IDs	Summary
Process	PRC-01..04	IP Helper map TCP (UDP best-effort); placeholder if unknown
Geo	GEO-01..06	Offline GeoLite2; private IP policy; home origin + override
Visual	VIS-01..05	3D globe, animated arcs, top-N cap for FPS
UX	UX-01..09	Dashboard layout, KPIs, feed, filters, pause, wizard
IPC	IPC-01..03	250–500 ms snapshots; start/stop/status/settings

3.4 Non-functional requirements

- PERF: Target 60 FPS under typical desktop traffic; throttle UI updates.
- REL: Capture failures surface actionable UI states.
- SEC: No payloads by default; no cloud telemetry; privilege explicit.
- USE: Core path start → arcs under 2 minutes after prerequisites.
- MAIN: Modular Rust modules; typed IPC; unit tests for flow/geo math.
- COMP: Windows 10 22H2+ and Windows 11 x64.

3.5 Acceptance criteria (v1)

1. With Npcap + admin + GeoIP, browsing produces visible arcs.
2. Major browser process names appear on enriched flows.
3. Hover shows process, location, rate; filters and pause work.
4. Missing Npcap/elevation yields clear remediation.
5. README + docs package present in repository.

4. System Architecture & Design (NV-SAD-001)

4.1 Architectural goals

Goal	Design response
Real-time feel	Event deltas every 250–500 ms; never per-packet UI
Silky 3D UI	WebGL globe isolated from capture thread
Safe capture	Header-only parse; privileged code in Rust
Extensibility	Modules: capture, flow, process, geo
Small footprint	Tauri instead of Electron

4.2 Context diagram

```
User → Network Visualizer (Tauri + React + Rust)
         ■■ Windows NIC + Npcap + IP Helper
         ■■ GeoLite2-City.mmdb (local)
         ■■ Public IP service (optional, home location)
```

4.3 Logical architecture

```
Presentation: React | Store | Globe | KPI | Feed | Filters
        ■ Tauri invoke + events
Application: commands + flow_snapshot emitter
Domain: Capture → Flow Aggregator ← Process Mapper
                → Geo Service
Infra: Npcap | IP Helper | maxminddb | optional HTTPS
```

4.4 Data pipeline

```
[NIC] → [Npcap] → [Parse L3/L4] → [Flow table]
  → [Rates + expiry] → [Process join] → [Geo enrich]
  → [Snapshot 250–500ms] → [React store] → [Globe + Feed + KPIs]
```

4.5 Flow aggregation algorithm

```
key = hash(src_ip, src_port, dst_ip, dst_port, protocol)
if new: insert first_seen = now
bytes_up/down += packet_len; packets += 1; last_seen = now
recompute rate_bps on interval (EMA / sliding window)
expire if now - last_seen > idle_timeout
```

4.6 Components

Component	Responsibility
Capture Engine	Open interface, read packets, extract 5-tuple + length
Flow Aggregator	Authoritative flow table, rates, expiry
Process Mapper	TCP/UDP owner tables → PID → process name
Geo Service	mmdb lookup, private IP policy, home location
Event Bridge	Compact snapshots to UI

Component	Responsibility
UI Store/Views	Filters, pause, arcs, feed, detail pin

4.7 Architecture Decision Records

ADR	Decision	Status
ADR-001	Tauri 2 over Electron	Accepted
ADR-002	Full Npcap capture; table fallback secondary	Accepted
ADR-003	3D globe default visualization	Accepted
ADR-004	Offline GeoLite2 over online-only APIs	Accepted
ADR-005	Header-only; no payload storage default	Accepted
ADR-006	Snapshot IPC not per-packet events	Accepted

5. Project Plan & Roadmap (NV-PLAN-001)

5.1 Delivery approach

Iterative phases with an early visual spike (Phase 1) to lock UX quality. Each phase ends with a demoable deliverable.

5.2 Phases

Phase	Name	Exit criteria
0	Scaffold & shell	Window + premium empty dashboard
1	Globe visual spike	Mock arcs wow pass
2	Capture engine	Live flows + rates in backend
3	Enrichment	Process + city/latlon on flows
4	Live wiring	Real traffic drives globe/panels
5	Polish & ship	Installer + wizard + QA

5.3 Milestones

ID	Milestone	Evidence
M0	Docs baseline	Docs/README present
M1	Visual prototype	Globe mock demo
M2	Capture alpha	Live flow events
M3	Enriched alpha	Process + geo fields
M4	Integrated beta	E2E dashboard
M5	v1 candidate	Installer + QA sign-off

5.4 Indicative effort (solo FTE days)

Phase	Estimate
0	0.5–1
1	1–2
2	2–4
3	1.5–3
4	2–3
5	1.5–3
Total	~9–16 days

5.5 Definition of done

1 Code builds without errors.

2. Behavior matches linked requirement IDs where applicable.
3. No payload logging introduced.
4. Failure paths are user-readable.
5. Docs/README updated if user-facing behavior changed.

6. Risk, Security & Privacy (NV-RISK-001)

6.1 Risk register

Score = Impact (1–5) × Likelihood (1–5).

ID	Risk	I	L	Sc	Mitigation
R-01	Admin/Npcap friction	5	4	20	Wizard + degraded mode
R-02	Globe jank at high flow count	4	3	12	Top-N arcs + throttle
R-03	GeoLite2 license misuse	4	2	8	Never commit mmdb
R-04	City geo inaccuracy	2	4	8	Approximate copy; show country
R-05	Unknown UDP process	3	4	12	Placeholder; still geo
R-06	High PPS CPU	4	2	8	Header-only + batching
R-07	AV / privilege perception	3	3	9	Sign builds; explain admin
R-08	Public IP lookup privacy	3	2	6	Document; allow override
R-09	Scope creep to SIEM	3	3	9	SRS non-goals fence
R-10	WebGL/driver issues	3	2	6	Document GPU requirements

6.2 Security principles

1. Least data — headers + counters only.
2. Local-first — no telemetry backend in v1.
3. Explicit privilege — never hide UAC need.
4. Fail closed on capture errors — no silent fake capture.
5. Dependency hygiene — pin versions; review native deps.

6.3 Privacy data categories

Category	Stored?	Leaves device?
Flow metadata (IP, ports, bytes, process)	Memory only (v1)	No
Geo results	Session cache	No
Settings	Local disk	No
Public IP query	Transient	Yes if used (optional)
Packet payload	Not stored	N/A

6.4 Residual risk statement

After mitigations, residual risk is acceptable for v1 personal/desktop use, dominated by setup friction (Npcap/admin) and approximate geolocation. Enterprise fleet use is not certified by this document.

7. Appendix

7.1 Technology stack

Area	Choice
Shell	Tauri 2
UI	React 18 + TypeScript + Vite
Globe	react-globe.gl (Three.js)
Capture	Npcap + Rust pcap
Parse	etherparse / pnet_packet
Process	windows crate / IP Helper
GeoIP	maxminddb + GeoLite2-City
State	Zustand (planned)

7.2 Glossary

Term	Definition
Flow	Logical conversation identified by 5-tuple
5-tuple	src IP/port, dst IP/port, protocol
Home location	Arc origin (public IP geo or override)
Arc	Visual link on the 3D globe
Enrichment	Adding process + geo metadata to a flow
Delta snapshot	Incremental UI update of flows

— End of Network Visualizer Project Documentation Package —